

Matrix Multiplication Optimization Project

Overview

This project implements and optimizes matrix multiplication followed by row-wise max subtraction. The task involves computing $C = A \times B$ for two square matrices, then subtracting the maximum value in each row from all elements in that row. This operation is commonly used in numerical stability optimizations, similar to the preprocessing step in softmax computation.

Task Description

The goal is to:

1. Implement matrix multiplication $C = A \times B$ for square matrices of size $M \times M$
2. For each row in the result matrix C , find the maximum value
3. Subtract this maximum from all elements in that row
4. Optimize the implementation for performance

Performance Testing:

- Test matrices of sizes: $M = \{256 \times x \mid x \in \{1, 2, 3, \dots, 10\}\}$
- Measure execution time for each optimization attempt
- Document improvements and analyze results

Environment Setup

Requirements

- **OS:** WSL (Windows Subsystem for Linux)
- **Tools:** Git, Makefile
- **CPU:** x86 architecture with AVX/AVX2 support
- **Compiler:** GCC with SIMD support flags

CPU Feature Verification

Ensure your CPU supports the required instruction sets:

```
# Check CPU flags
cat /proc/cpuinfo | grep flags

# Required flags: avx, avx2, fma, sse, sse2
```

Compilation

```
# For AVX2 implementations (opt_v6 onwards)
g++ -mavx2 -mfma -O3 opt_vX.cpp -o opt_vX
```

```
# For SSE implementations (opt_v5)
g++ -msse4.1 -O3 opt_v5.cpp -o opt_v5

# For basic implementations
g++ -O3 naive.cpp -o naive
```

Optimization Journey

Version 1: Naive Implementation ([naive.cpp](#))

Description:

- Standard triple-nested loop matrix multiplication
- Row-by-row max finding and subtraction
- No optimization techniques applied

Performance: Baseline - quite slow

Key Characteristics:

- Straightforward implementation
 - Poor cache utilization
 - No instruction-level parallelism
-

Version 2: Register Optimization ([opt_v2.cpp](#))

Optimization Technique:

- Load `C[i][j]` into a register variable before the innermost loop
- Avoids repeated memory accesses to `c[i][j]` during the k-loop
- Use pointer arithmetic for better code generation

Theory:

- Array elements must reside in memory
- Local scalar variables can be placed in CPU registers
- Register access is much faster than memory access

Performance: Theoretically should improve, but minimal practical gain observed

Reason: Modern compilers often perform this optimization automatically

Version 3: Register Blocking (2×2) ([opt_v3.cpp](#))

Optimization Technique:

- Process 2×2 blocks of matrix C simultaneously
- Load a 2×1 slice of A and a 1×2 slice of B
- Perform outer-product rank-1 update on the 2×2 C block

Benefits:

- Better data reuse in registers
- Reduces memory traffic
- Improves instruction-level parallelism

Performance: Great improvement over naive implementation

Mathematical View:

```
C[i:i+1][j:j+1] += A[i:i+1][k] × B[k][j:j+1]
```

Version 4: Enhanced Register Blocking (4×4) ([opt_v4.cpp](#))**Optimization Technique:**

- Extend blocking to 4×4 blocks
- Process larger chunks per iteration
- More register reuse opportunities

Performance: Further improvement due to increased register blocking

Trade-off:

- Requires more registers
- Better for larger matrices

Version 5: SIMD with SSE ([opt_v5.cpp](#))**Optimization Technique:**

- Use Single Instruction Multiple Data (SIMD) instructions
- Process 4 float elements simultaneously using `__m128` (128-bit vectors)
- 4×4 block processing with SIMD

Key Features:

- Vectorized operations: `_mm_mul_ps`, `_mm_add_ps`
- Broadcast scalar values to vectors: `_mm_set1_ps`
- Aligned memory access where possible

Performance: Significant improvement due to parallel processing

SIMD Benefits:

- Single instruction processes multiple data elements
- Better cache utilization
- Reduced instruction overhead

Version 6: AVX2 with 4×8 Blocking ([opt_v6.cpp](#))

Optimization Technique:

- Upgrade from `__m128` to `__m256` (256-bit vectors)
- Process 8 float elements per lane
- 4×8 register blocking
- Loop unrolling by 4 iterations
- Use fused multiply-add: `_mm256_fmadd_ps` (single instruction for mul+add)

Key Improvements:

- `_mm256_fmadd_ps`: Fused multiply-add reduces instruction count
- Loop unrolling: Reduces loop overhead
- Better register utilization

Performance: Slight improvement over Version 5

Note: Requires AVX2 and FMA instruction sets

Version 7: 8×4 Blocking ([opt_v7.cpp](#))

Optimization Technique:

- Change blocking from 4×4 to 8×4
- Better utilization of all 256-bit YMM registers
- Process 8 rows and 4 columns simultaneously

Performance: Improved due to more comprehensive blocking

Register Usage:

- 8 accumulator registers (c0-c7) for 8 rows
 - Better parallelism
-

Version 8: Cache Blocking ([opt_v8.cpp](#))

Optimization Technique:

- Implement multi-level cache blocking
- Structure: `matmul_v8 (cache blocking) + macro_kernel + scalar_kernel`

Blocking Parameters:

```
M_BLOCKING: 192  (row blocking)
N_BLOCKING: 2048 (column blocking)
K_BLOCKING: 384  (inner dimension blocking)
```

Architecture:

- **Macro Kernel:** High-performance 8×8 block computation
- **Scalar Kernel:** Handles edge cases within blocks
- **Cache Blocking:** Ensures data fits in L2/L3 cache

Benefits:

- Reduces cache misses
- Better temporal and spatial locality
- Handles large matrices efficiently

Performance: Significant improvement for large matrices

Version 9: Packing + Cache Blocking (opt_v9.cpp)

Optimization Technique:

- Data packing/re-layout into continuous buffers
- Eliminates performance penalty from irregular memory access in `vector<vector>`
- 8×8 AVX2 macro kernel with packed data

Packing Strategy:

- **Packing A:** Transpose and pack 8 rows × K columns into linear memory
 - Makes `A[i][k]...A[i+7][k]` physically adjacent
 - Enables fast broadcasting in kernel
- **Packing B:** Linear copy of K rows × 8 columns
 - Ensures completely contiguous access
 - Avoids row-jumping overhead in `vector<vector>`

Key Features:

- `packing_a_k9()`: Packs A matrix blocks
- `packing_b_k9()`: Packs B matrix blocks
- `macro_kernel_k9()`: Computes on packed buffers
- `scalar_kernel_k9()`: Handles edge cases

Performance: Fast! Best performance achieved

Trade-off:

- Code complexity increases significantly
 - Deeply tied to hardware layer (cache, TLB, memory, pointers)
 - Requires careful memory management
-

Performance Analysis

Optimization Techniques Summary

Version	Key Technique	Performance Gain
---------	---------------	------------------

Version	Key Technique	Performance Gain
v1 (naive)	Baseline	1.0x
v2	Register variable	~1.0x (minimal)
v3	2×2 Blocking	Significant
v4	4×4 Blocking	Good
v5	SIMD (SSE)	Great
v6	AVX2 + FMA	Better
v7	8×4 Blocking	Improved
v8	Cache Blocking	Significant (large matrices)
v9	Packing + Cache	Best

Key Insights

- 1. **Register Blocking**: Fundamental optimization that enables others
- 2. **SIMD**: Major performance boost through parallelism
- 3. **Cache Blocking**: Critical for large matrices
- 4. **Data Packing**: Eliminates memory access penalties
- 5. **Fused Operations**: Reduces instruction count

Code Structure

```
self-solution/code/
├─ naive.cpp      # Baseline implementation
├─ opt_v2.cpp     # Register optimization
├─ opt_v3.cpp     # 2×2 blocking
├─ opt_v4.cpp     # 4×4 blocking
├─ opt_v5.cpp     # SIMD (SSE)
├─ opt_v6.cpp     # AVX2 + FMA
├─ opt_v7.cpp     # 8×4 blocking
├─ opt_v8.cpp     # Cache blocking
└─ opt_v9.cpp     # Packing + cache blocking
```

Running the Code

Basic Usage

```
# Compile
g++ -mavx2 -mfma -O3 opt_v9.cpp -o opt_v9

# Run
./opt_v9
```

Testing Different Versions

```
# Test all versions
for v in {2..9}; do
    echo "Testing opt_v$v"
    g++ -mavx2 -mfma -O3 opt_v$v.cpp -o opt_v$v
    ./opt_v$v
done
```

Technical Details

Memory Access Patterns

Naive Implementation:

- Poor spatial locality
- Cache misses on every iteration
- No data reuse

Optimized (v9):

- Packed data ensures contiguous access
- Cache blocking maintains temporal locality
- Register blocking maximizes data reuse

SIMD Instructions Used

- `_mm256_setzero_ps()`: Initialize vector to zero
- `_mm256_load_ps()` / `_mm256_loadu_ps()`: Load aligned/unaligned data
- `_mm256_store_ps()` / `_mm256_storeu_ps()`: Store aligned/unaligned data
- `_mm256_broadcast_ss()`: Broadcast scalar to vector
- `_mm256_fmadd_ps()`: Fused multiply-add
- `_mm256_add_ps()`: Vector addition

Cache Hierarchy Considerations

- **L1 Cache**: ~32KB, fastest, used for register blocking
- **L2 Cache**: ~256KB-1MB, used for cache blocking
- **L3 Cache**: ~8-32MB, shared across cores
- **Main Memory**: Slowest, minimize accesses

References

- [Optimizing DGEMM on Intel CPUs with AVX512F](#)
- Intel Intrinsics Guide: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/>
- BLIS Framework: High-performance BLAS implementation

Conclusion

This project demonstrates the progression from naive matrix multiplication to highly optimized implementations using:

- Register blocking for data reuse
- SIMD instructions for parallelism
- Cache blocking for memory hierarchy optimization
- Data packing for memory access pattern optimization

The final implementation (v9) achieves significant performance improvements through careful consideration of hardware characteristics and memory access patterns.